German Traffic Sign Classification with Optimized Convolutional Neural Networks: Pruning and Quantization for Efficient Deployment:

1. Executive Summary

This report details a deep learning project focused on the classification of German Traffic Signs. The project utilizes a custom Convolutional Neural Network (CNN) and rigorously investigates advanced model optimization techniques, including dataset balancing through extensive augmentation, **L1 unstructured pruning**, and **Quantization Aware Training (QAT)**. The primary objective is to achieve an optimal balance between high classification accuracy and significantly reduced model efficiency, encompassing both model size and inference latency. This optimization effort aims to produce models highly suitable for deployment in resource-constrained environments, such as embedded systems or mobile applications, a critical requirement for real-world machine learning solutions.

2. Introduction

Accurate and efficient traffic sign recognition is a pivotal component of advanced driver-assistance systems (ADAS) and autonomous vehicles. The ability to identify traffic signs in real-time under diverse environmental conditions is crucial for safe navigation. While deep learning, particularly Convolutional Neural Networks (CNNs), has demonstrated state-of-the-art performance in image classification tasks, the high computational demands and large model sizes often limit their deployment on edge devices with restricted resources.

This project addresses these challenges by developing a robust traffic sign classification system based on a custom CNN architecture. Furthermore, it systematically applies and evaluates advanced model optimization techniques—specifically **L1 unstructured pruning** and **Quantization Aware Training (QAT)**—to significantly reduce model complexity and improve inference efficiency without substantial loss in classification accuracy. This report outlines the methodology, experimental setup, and comprehensive results of these optimization efforts.

3. Methodology

This section provides a detailed account of the dataset utilized, the data preparation and augmentation strategies employed, the design of the custom CNN architecture, and the step-by-step methodology for model optimization.

3.1. Dataset Description

The project utilizes the **German Traffic Sign Recognition Benchmark (GTSRB)** dataset, a widely recognized industry-standard benchmark for traffic sign classification.

- **Classes:** The dataset comprises **43 distinct class labels**, each representing a unique German traffic sign (e.g., various speed limits, warning signs, mandatory instructions).
- **Annotations:** Alongside image files, the dataset provides comprehensive CSV metadata files (Train.csv and Test.csv). These files are critical as they include ClassId labels for each image and precise **Region of Interest (ROI) coordinates** that define the bounding box of each traffic sign within its respective image.
- **Original Imbalance:** The raw GTSRB dataset exhibits significant class imbalance. The **largest class originally contains approximately 2,250 images**, while the **smallest class has only around 69 images**. This disparity poses a considerable challenge for training a model that performs robustly across all classes without specific balancing strategies.
- **Total Images (Original):** The original training dataset consists of **over 39,000 images**.
- **Image Dimensions:** The images within the dataset vary in resolution, ranging from a minimum of **23x23 pixels up to approximately 150x150 pixels**. For consistency in model input, all images are uniformly resized to 64x64 pixels prior to being fed into the neural network.
- **External Test Set:** A dedicated, independent external test dataset is provided, consisting of **12,630 images**. These images also possess variable dimensions similar to the training set and come with their own label CSV file. This separate test set enables a robust and unbiased evaluation of the model's generalization capabilities on unseen data.

3.2. Data Preparation and Augmentation

To effectively counteract the inherent class imbalance and enhance the model's ability to generalize to diverse real-world conditions, a strategic data augmentation pipeline was implemented.

1. **Targeted Balancing:** The core objective of the augmentation process was to achieve a perfectly balanced training dataset. This implies that after augmentation, **every one of the 43 classes contains exactly 2,250 images**. This target count was established based on the number of images in the largest class found in the original dataset.
2. **Augmentation Strategy:** For any class that contained fewer than the 2,250 target images, new augmented images were generated by randomly transforming existing images from

that specific class. A key and deliberate aspect of this approach was to apply augmentations exclusively to the **Region of Interest (ROI)** of the traffic sign. This technique preserved the original image background context while introducing variations only to the sign itself, enhancing realism and preventing the model from learning artifacts from background augmentations.

3. **Augmentation Techniques:** The following specific transformations were applied to the extracted ROIs:
    - **Color Jitter:** Randomly adjusts brightness, contrast, saturation, and hue (using parameters like 0.2, 0.2, 0.2, 0.1 for variation). This simulates varying lighting conditions and helps the model become invariant to illumination changes.
    - **Affine Transformations (Translate):** Randomly shifts the image horizontally and vertically (e.g., with a translation range of 0.1 of image width/height). This mimics slight camera movements or minor variations in sign positioning within the frame.
    - **Gaussian Blur:** Applies a Gaussian blur (e.g., with kernel size (3, 3) and sigma range (0.1, 2.0)). This simulates varying atmospheric conditions (e.g., fog, haze) or slight motion blur, enhancing the model's robustness to image quality degradation.
4. **Integration & Final Dataset Size:** After augmentation, the transformed ROIs were seamlessly reintegrated back into their original image backgrounds. These newly generated images were then saved, and the training metadata file was updated to include paths to both original and augmented images. Post-augmentation, the final training dataset size reached a substantial **96,750 images**.

Result of Data Balancing:

Following the data augmentation process, the training dataset achieved perfect class balance, with each of the 43 classes containing precisely 2,250 images.

3.3. Model Architecture: TrafficSignCNN

A custom Convolutional Neural Network, named TrafficSignCNN, was meticulously developed specifically for this traffic sign classification task. Its architecture is carefully designed to strike a balance between computational efficiency and strong representational capacity, making it suitable for optimization efforts.

- **Sequential Convolutional Blocks:** The core of the network is composed of four sequential conv_block units. Each conv_block thoughtfully encapsulates a Conv2d layer for effective feature extraction, immediately followed by BatchNorm2d for stable training and reduced overfitting, ReLU activation for introducing non-linearity, and MaxPool2d for spatial downsampling. The channel progression through these blocks is designed to progressively increase feature complexity: [32, 64, 128, 256].
- **Global Average Pooling:** Subsequent to the convolutional layers, a global average pooling layer is applied. This layer reduces each feature map to a single value, contributing to

making the model more robust to spatial translations and significantly reducing the number of parameters required in the subsequent fully connected layer.

- **Dropout:** A dropout layer is strategically applied before the final classification layer. This technique randomly sets a fraction of input units to zero during training, which is a common and effective method to prevent overfitting.
- **Linear Classifier:** A final linear layer outputs the probabilities for the 43 distinct traffic sign classes, providing the model's prediction.
- **Quantization Readiness:** The model's design inherently includes features to facilitate seamless quantization. It incorporates QuantStub and DeQuantStub layers at the input and output respectively, explicitly marking the boundaries for quantization. Furthermore, it includes a fuse_model method to fuse Conv-BatchNorm-ReLU operations, a crucial pre-processing step for effective and hardware-accelerated integer quantization.

3.4. Model Optimization Pipeline

The entire model optimization process follows a well-defined, sequential pipeline to systematically improve efficiency:

1. **Baseline Model Training:** Establishment of a performance baseline with an unoptimized, full-precision model.
2. **Model Pruning:** Strategic reduction of model redundancy and size based on identified optimization opportunities.
3. **Quantization Aware Training (QAT):** Further optimization of the pruned model to enable efficient, lower-precision inference with minimal accuracy impact.

Two distinct experimental approaches were undertaken to thoroughly evaluate the impact and effectiveness of the pruning strategy within this pipeline:

3.4.1. Baseline Model Training

The TrafficSignCNN was initially trained as a full-precision (FP32) baseline model using the augmented dataset. The dataset was judiciously split, allocating 70% for training, 15% for validation, and 15% for independent testing.

- **Optimizer:** The Adam optimizer was employed, initialized with a learning rate of 0.001.
- **Loss Function:** CrossEntropyLoss was utilized as the loss function, which is highly suitable for multi-class classification problems.
- **Epochs:** The model was trained for 15 epochs to ensure adequate convergence and performance stabilization.

Baseline Model Performance:

Model	Parameters	Test Accuracy	Size (MB)
Baseline	399,947	0.9974	1.61

3.4.2. Model Pruning Experiments

Prior to initiating the pruning process, a detailed analysis of the full-precision baseline model was conducted. This systematic analysis aimed to identify specific layers and parameters that offered the most promising opportunities for reduction without causing a significant degradation in the model's overall performance.

Analysis for Optimization Opportunities:

- **BN Scaling Analysis:** This analysis meticulously examines the gamma (weight) values within Batch Normalization layers. Channels associated with consistently low gamma values indicate a diminished contribution to the feature representation, marking them as prime candidates for removal during pruning.
- **Filter Redundancy Analysis:** This technique identifies convolutional filters that exhibit high similarity to other filters or possess very low L1-norms. Such characteristics suggest redundancy, implying these filters can be safely eliminated to reduce model size and computational load.
- **Activation Utility Analysis:** By measuring the mean absolute activation of each channel across a representative dataset, this analysis identifies channels with consistently low activation magnitudes. These channels are deemed less important for the network's function and are thus targeted for pruning.
- **Parameter Distribution Analysis:** Examination of the statistical distribution of model weights can reveal inherent sparsity or clusters of values located near zero. This provides valuable insights into the potential for fine-grained pruning or the applicability of quantization techniques.
- **Output Shape Recording:** Tracking the dimensional changes of tensors as they pass through each layer provides crucial information. This is particularly important for structural pruning, ensuring that pruned models maintain compatible layer interfaces and avoid dimension mismatches.

Summary of Analysis Findings:

The comprehensive analysis consistently highlighted significant channel pruning potential across all convolutional layers within the TrafficSignCNN. Approximately 34-36% of channels were explicitly identified as low-utility candidates. These data-driven insights directly informed the specific pruning percentages applied in our primary, analysis-driven experiment.

Experiment 1: Analysis-Driven Pruning (Main Pipeline)

In this experiment, L1 unstructured pruning was strategically applied based on the specific recommendations derived from the detailed model analysis. This involved pruning a variable percentage of channels in each layer, precisely corresponding to their identified redundancy or low utility. Following the pruning operation, the model underwent a critical single-epoch fine-tuning phase. This short fine-tuning step was essential to recover any minor accuracy degradation

that might have been introduced by the removal of parameters, ensuring the model's performance remained robust.
The pruning configuration targeted specific channel reductions based on analysis:
Original channel dimensions: [32, 64, 128, 256]
Pruned channel dimensions: [21, 41, 82, 164]
The final pruned model, which subsequently served as the input for the quantization step in the main optimization pipeline, achieved the following characteristics:

Model	Parameters	Test Accuracy	Size (MB)
Pruned	167,317	0.9974	0.68

Experiment 2: Fixed-Percentage Pruning for Performance Observation

To gain a comprehensive understanding of the relationship between pruning intensity and overall model performance, a second, exploratory experiment was conducted. In this experiment, pruning was deliberately applied uniformly across all layers at predefined fixed percentages: 10%, 30%, 50%, and 90%. This systematic approach allowed for a direct observation of performance trends, including potential accuracy degradation and efficiency gains, as the model size was progressively reduced.

Pruning %	Parameters	Test Accuracy	Size (MB)	GMACs (Giga MACs)
10%	324,798	0.9966	1.31	0.013
30%	199,356	0.9974	0.81	0.008
50%	103,227	0.9908	0.42	0.004
90%	5,244	0.2207	0.03	0.000

This experiment was instrumental in confirming that the **30% pruning level yielded an optimal trade-off**. At this level, the model remarkably maintained near-baseline accuracy (0.9974) while simultaneously achieving a significant reduction in parameters and model size. Beyond the 50% pruning threshold, the accuracy degradation became substantial, with a particularly pronounced drop at the 90% pruning level.

3.4.3. Quantization Aware Training (QAT)

Following the pruning step, the project employs **Quantization Aware Training (QAT)**, a sophisticated optimization technique that integrates quantization directly into the training loop itself. Unlike simpler methods like Post-Training Static Quantization (PTQ Static), QAT allows the model to actively "learn" to operate effectively with lower-precision (e.g., INT8) weights and activations during the fine-tuning phase. This adaptive learning process consistently leads to significantly better accuracy retention, frequently matching or even surpassing the performance of the full-precision baseline model.

Methodology for QAT:

1. **Module Preparation:** The already pruned, full-precision model is meticulously prepared for QAT. This critical preparation involves two main steps:
    - **Module Fusion:** Sequential layers such as Conv-BatchNorm-ReLU operations are "fused" into single, more efficient operational units. This process is vital as it reduces computational overhead and is a crucial pre-processing step for effective integer quantization on various hardware platforms.
    - **Quant/DeQuant Stubs:** QuantStub and DeQuantStub layers are strategically inserted at the input and output boundaries of the quantizable parts of the network, respectively. These stubs explicitly define the points where data is converted from FP32 (floating-point 32-bit) to INT8 (integer 8-bit) and then back again, ensuring that the bulk of internal computations occurs in the more efficient INT8 format.
    - **Quantization Observers:** PyTorch's quantization observers are placed throughout the model. Crucially, in QAT, these observers are active throughout the fine-tuning process. They simulate the actual quantization (e.g., rounding to INT8 values) in the forward pass, and, importantly, they propagate gradients through these approximate operations during the backward pass.
2. **Fine-tuning with Simulated Quantization:** The prepared model (which now includes observers that simulate the effects of quantization) is then fine-tuned for a few additional epochs using the training dataset. During this fine-tuning period, the model's weights and activations are constantly exposed to the "noise" or effects introduced by the simulated lower-precision quantization. This iterative exposure allows the network to adapt its parameters and effectively learn robustness to these lower-precision operations, thereby minimizing the accuracy degradation often associated with post-training quantization.
3. **Final Conversion:** Once the QAT fine-tuning is successfully completed, the model's floating-point weights and biases, along with the calibrated activation ranges learned during training, are converted to their final, compact INT8 representation. This INT8 model is then ready for deployment, offering significant advantages in terms of size and computational requirements.

4. Results and Performance

The comprehensive optimization pipeline successfully reduced model size and computational complexity while preserving high classification accuracy. All inference prediction results presented below were obtained on the **12,630 images from the independent external test dataset.**

4.1. Overall Model Performance Metrics (Accuracy, Precision, Recall, F1 Score)

Model	Accuracy (%)	Precision (%)	Recall (%)	F1 Score (%)
Baseline	97.65	97.71	97.65	97.59
Pruned	97.18	97.32	97.18	97.12
Quantized	96.56	96.79	96.56	96.49

4.2. Performance Metrics (Inference Time, FPS, GMACs, Model Size)

Model	Avg Inference Time (ms/img)	Total Inference Time (ms)	FPS (Frames/sec)	GMACs (Giga MACs)	Model Size (MB)
Baseline	0.9704	12255.91	1030.52	0.015422	1.5381
Pruned	0.9611	12138.22	1040.52	0.006685	0.6489
Quantized	2.7998	35360.98	357.17	0.000081	0.1678

4.3. Key Findings and Discussion:

- **Critical Inference Environment Detail:** It is essential to highlight the specific hardware environment used for inference time measurements. The **Baseline and Pruned models were evaluated on a GPU**, leveraging their full-precision capabilities and parallel processing power. In contrast, the **Quantized model was specifically run on a CPU**. This disparity in hardware execution significantly impacts the observed inference times, explaining why the quantized model exhibits a lower Frames Per Second (FPS) in the provided table. On hardware with dedicated INT8-accelerated units (such as modern GPUs with Tensor Cores or specialized Neural Processing Units/NPUs), the quantized model would typically demonstrate substantial speedups due to its extremely low computational requirements (GMACs).
- **Quantization Speed on CPU vs. Specialized Hardware:** A critical insight gained through this work is that while architectural downsizing and parameter reduction are vital for efficiency, the raw inference time of a quantized model, particularly when executed on a general-purpose CPU, can sometimes appear to increase compared to an optimized full-precision model. This can be attributed to the inherent overhead involved in the quantization/dequantization process and the specific CPU optimizations available for integer operations. However, the dramatic reduction in GMACs remains a core benefit, signifying significantly lower actual computational work and enabling superior performance on hardware designed for integer arithmetic.
- **Superiority of Analysis-Driven Pruning:** The experimental results unequivocally demonstrate that **following the analysis-driven pruning process yields superior performance** (maintaining accuracy almost identical to the baseline) compared to using

arbitrary fixed-percentage pruning. This outcome strongly validates the importance and effectiveness of employing intelligent, data-driven pruning strategies.

- **Exceptional Accuracy Preservation:** Both the pruning step and the subsequent Quantization Aware Training were successfully applied with **minimal degradation in classification accuracy**. The analysis-driven pruned model achieved a near-identical accuracy to the baseline (97.18% versus 97.65%). Furthermore, the QAT-quantized model maintained a remarkably strong performance (96.56%). This collectively underscores the effectiveness of both optimization techniques in preserving overall model quality.
- **Dramatic Model Size Reduction:**
    - Pruning alone effectively reduced the model size from **1.61 MB** (for the baseline model) to **0.65 MB** (for the pruned model), representing a substantial **~60% reduction**. This significantly benefits memory footprint and accelerates model loading times.
    - **The combination of pruning and QAT achieved extraordinary model size compression.** The QAT-quantized model was further reduced to an impressive **0.17 MB**. This marks a staggering **~90% reduction** in size compared to the original baseline model. This level of compression makes the model exceptionally lightweight and ideal for deployment on highly resource-constrained devices, mobile applications, or scenarios demanding rapid model distribution and low storage requirements.
- **Significant Computational Efficiency (GMACs) Gains:**
    - Pruning substantially reduced the GMACs (Giga Multiply-Accumulate Operations) from 0.0154 (baseline) to 0.0067, indicating a significant cut in the floating-point computational load.
    - QAT pushed the GMACs down to an incredibly low **0.000081 GMACs**. This metric profoundly showcases the extreme efficiency gained by enabling highly optimized integer arithmetic. Such a dramatic reduction is critical for achieving lower power consumption and higher throughput on hardware specifically optimized for integer operations.

5. Conclusion

This project successfully developed and optimized a Convolutional Neural Network for German traffic sign classification. Through a strategic pipeline involving comprehensive data augmentation, analysis-driven L1 unstructured pruning, and Quantization Aware Training, the model's efficiency was dramatically enhanced. The findings demonstrate that it is possible to achieve substantial reductions in model size (approximately 90%) and computational cost (GMACs) with minimal compromise on classification accuracy (less than 1% drop from baseline).

While observed inference speeds for the quantized model on CPU showed overheads, the significant GMACs reduction confirms its superior computational efficiency, particularly for deployment on dedicated integer-accelerated hardware. This comprehensive optimization approach renders the developed models highly viable for practical, real-world applications in resource-constrained environments, contributing to more efficient and scalable machine learning solutions in autonomous systems.

6. Code Availability and Reproducibility

The source code, including the custom CNN architecture, data processing scripts, and implementation of pruning and quantization techniques, is provided to ensure full reproducibility of the reported results. Detailed instructions for setting up the environment and executing the experiments are included within the project's documentation.

7. Contact Information

For any inquiries, clarifications, or further discussions regarding this project, please feel free to contact:

Suraj Varma
sv3677781@gmail.com